Lecture 17 – Compiling with Continuations

COSE212: Programming Languages

Jihyeok Park



2026 Fall

Recall

- A **continuation** is the **rest of the computation**.

- So far, we learned:
  - An interpreter of FAE in **CPS** (Lecture 14)
  - Small-step semantics with **continuations** (Lecture 15)
  - **First-class continuations** in KFAE (Lecture 16)

Recall

- A **continuation** is the **rest of the computation**.
- So far, we learned:
  - An interpreter of FAE in **CPS** (Lecture 14)
  - Small-step semantics with **continuations** (Lecture 15)
  - **First-class continuations** in KFAE (Lecture 16)
- Today, let's **compile** FAE into a **low-level IR** like assembly, using **continuations**.

Contents

1. Compilers

2. Expressions

3. Functions
   - Lambda Lifting
   - Closure Conversion

4. Names and Registers
   - Alpha Renaming
   - Register Allocation
   - Optimization

5. First-Class Continuations

Contents

1. Compilers

2. Expressions

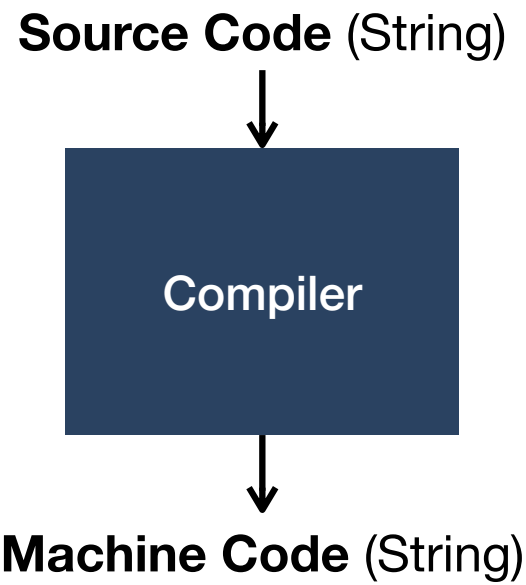
3. Functions
   - Lambda Lifting
   - Closure Conversion

4. Names and Registers
   - Alpha Renaming
   - Register Allocation
   - Optimization

5. First-Class Continuations

Compilers

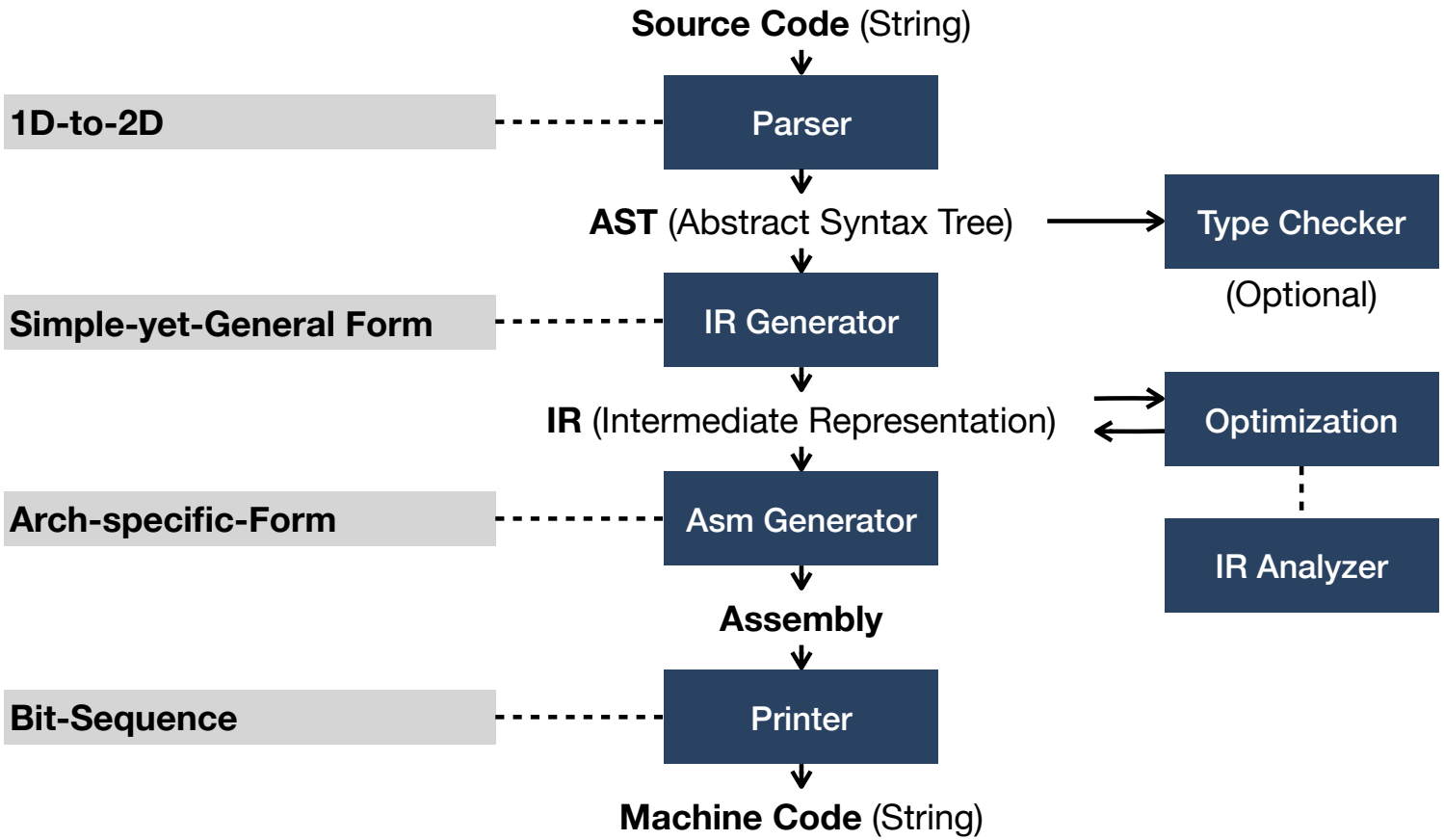
A **compiler** is a program that translates a program written in one language (the **source language**) into an equivalent program in another language (the **target language**).



Typically, the source language is a **high-level language** (e.g., Scala, Python, JavaScript, etc.) and the target language is a **low-level language** (e.g., JVM bytecode, LLVM IR, assembly, etc.).

Compilers

The following figure shows a typical compilation process:



Let's focus on the **IR Generator** to learn how to compile with functional languages with continuations into a **low-level IR**.

Our Target: A Low-level IR

PLRG

An **intermediate representation (IR)** is what a compiler translates a program into before producing machine code. Ours is **assembly-like**:

Instruction	Meaning
`mov d, s`	copy `s` to `d`
`add d, s1, s2`	`d` $\leftarrow$ `s1` $+$ `s2`
`mul d, s1, s2`	`d` $\leftarrow$ `s1` $\times$ `s2`
`jmp s`	jump to the label (or the address in) `s`
`lt d, s1, s2`	`d` $\leftarrow 1$ if `s1` $<$ `s2`, otherwise 0
`jz s, L`	jump to `L` if `s` is 0
`ld d, s, i`	`d` $\leftarrow$ memory at address `s` $+$ `i` (**load**)
`st d, i, s`	memory at address `d` $+$ `i` $\leftarrow$ `s` (**store**)
`L:`	a **label**: a name for the address of the next line

Our Target: A Low-level IR

PLRG

An **intermediate representation (IR)** is what a compiler translates a program into before producing machine code. Ours is **assembly-like**:

Instruction	Meaning
`mov d, s`	copy `s` to `d`
`add d, s1, s2`	`d` $\leftarrow$ `s1` $+$ `s2`
`mul d, s1, s2`	`d` $\leftarrow$ `s1` $\times$ `s2`
`jmp s`	jump to the label (or the address in) `s`
`lt d, s1, s2`	`d` $\leftarrow 1$ if `s1` $<$ `s2`, otherwise 0
`jz s, L`	jump to `L` if `s` is 0
`ld d, s, i`	`d` $\leftarrow$ memory at address `s` $+$ `i` (**load**)
`st d, i, s`	memory at address `d` $+$ `i` $\leftarrow$ `s` (**store**)
`L:`	a **label**: a name for the address of the next line

- An operand is a **register** (`r1`, `r2`, ...), a **number**, or a **label**.
- A program starts at `START` and stops when it reaches `HALT`, an empty label at the end.

Our Target: An Example

```
START:
  add x1, 1, 2
  mov r1, x1
  jmp HALT
HALT:
```

- Start at `START`.
- `add x1, 1, 2`: compute $1 + 2$ into `x1`.
- `mov r1, x1`: put the result 3 in `r1`.
- `jmp HALT`: jump to `HALT` and stop.

Compiling Functional Languages

How to compile FAE into this IR? Each feature of FAE that the IR does not have needs its own step:

FAE	**Low-level IR**	**Step**
nested expressions	one operation each	**ANF**
branches (`if`)	conditional jumps	**ANF**, labels
functions and calls	labels and jumps	**CPS**, **lambda lifting**
captured variables	arrays in memory	**closure conversion**
unlimited names	a few registers	**register allocation**
first-class continuations	a label to jump to	nothing more

Let's see why each step is needed, with a **small example** for each.

Contents

1. Compilers

2. Expressions

3. Functions
   - Lambda Lifting
   - Closure Conversion

4. Names and Registers
   - Alpha Renaming
   - Register Allocation
   - Optimization

5. First-Class Continuations

A-Normal Form (ANF)

PLRG

An IR instruction does **one** operation, but FAE expressions are **nested**. In the **A-normal form (ANF)**, every operand is a **variable** or a **constant**:

```
1 + 2 * 3
```

A-Normal Form (ANF)

An IR instruction does **one** operation, but FAE expressions are **nested**. In the **A-normal form (ANF)**, every operand is a **variable** or a **constant**:

```
1 + 2 * 3
```

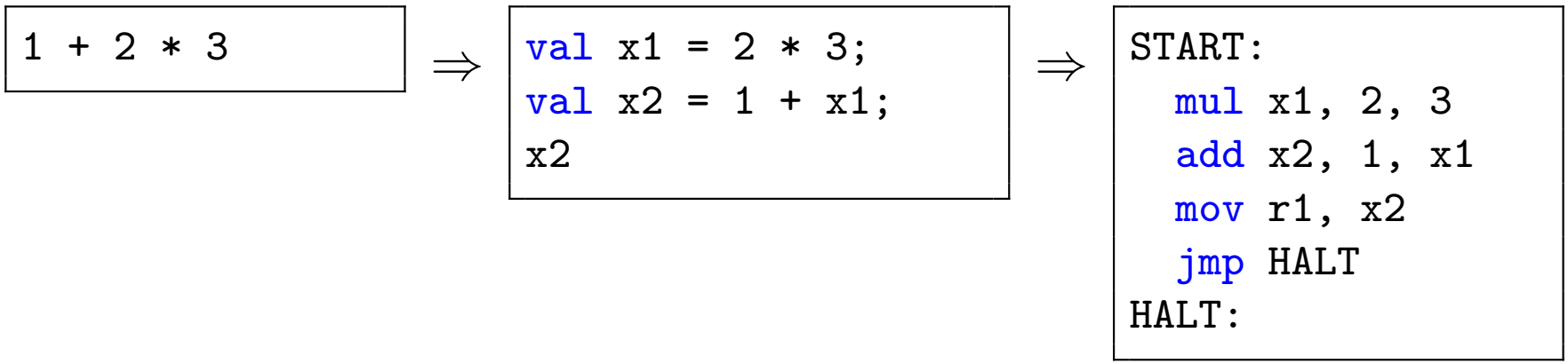
$\Rightarrow$

```
val x1 = 2 * 3;
val x2 = 1 + x1;
x2
```

A-Normal Form (ANF)

PLRG

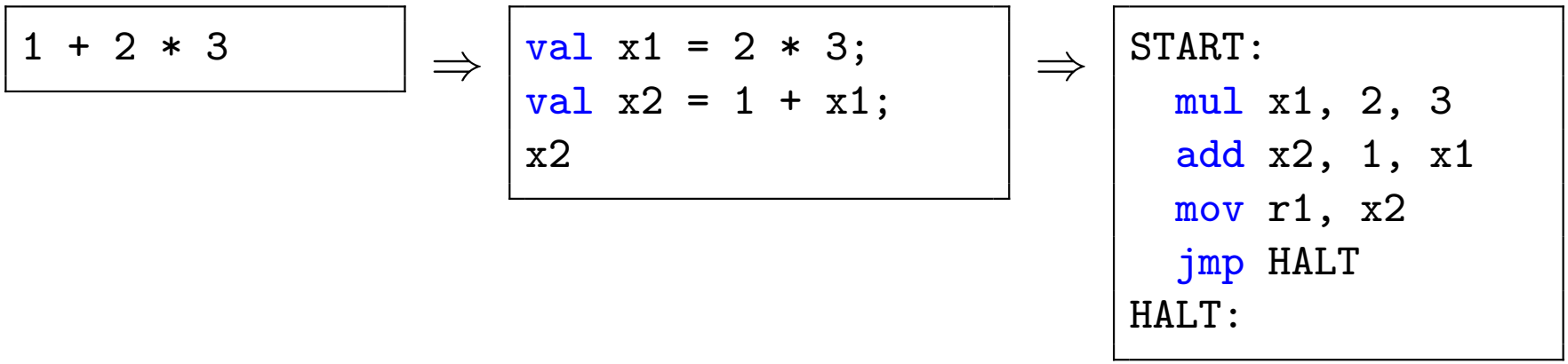
An IR instruction does **one** operation, but FAE expressions are **nested**. In the **A-normal form (ANF)**, every operand is a **variable** or a **constant**:



A-Normal Form (ANF)

PLRG

An IR instruction does **one** operation, but FAE expressions are **nested**. In the **A-normal form (ANF)**, every operand is a **variable** or a **constant**:

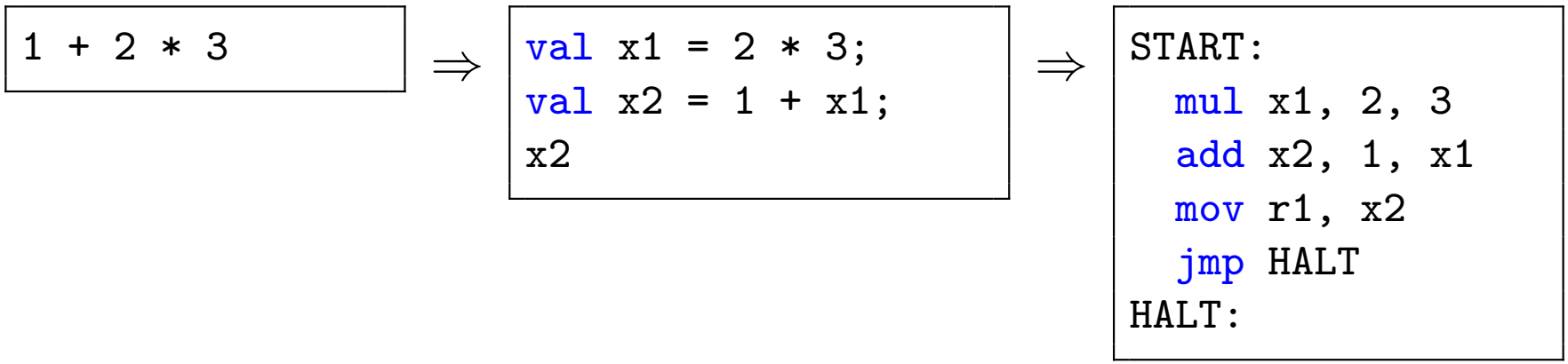


ANF gives every intermediate value a **name** and fixes the **order**. Then, each line is **one instruction**, and we leave the result in `r1`.

A-Normal Form (ANF)

PLRG

An IR instruction does **one** operation, but FAE expressions are **nested**. In the **A-normal form (ANF)**, every operand is a **variable** or a **constant**:



ANF gives every intermediate value a **name** and fixes the **order**. Then, each line is **one instruction**, and we leave the result in `r1`.

In the IR, we keep the names `x1`, `x2`, ... for now, as many as we need. They become registers at the end.

A-Normal Form (ANF)

An IR instruction does **one** operation, but FAE expressions are **nested**. In the **A-normal form (ANF)**, every operand is a **variable** or a **constant**:

```
1 + 2 * 3
```

⇒

```
val x1 = 2 * 3;
val x2 = 1 + x1;
x2
```

⇒

```
START:
  mul x1, 2, 3
  add x2, 1, x1
  mov r1, x2
  jmp HALT
HALT:
```

ANF gives every intermediate value a **name** and fixes the **order**. Then, each line is **one instruction**, and we leave the result in `r1`.

In the IR, we keep the names `x1`, `x2`, ... for now, as many as we need. They become registers at the end.

For **arithmetic**, this is enough! What about **branches**?

Branches

Let's extend FAE with `if (e) e else e` and `<`:

```
(if (1 < 2) 1 + 2 else 3 * 4) * 5
```

Branches

Let's extend FAE with `if (e) e else e` and `<`:

```
(if (1 < 2) 1 + 2 else 3 * 4) * 5
```

```
val x1 = 1 < 2;
val x2 = if (x1) {
  val x3 = 1 + 2;
  x3
} else {
  val x4 = 3 * 4;
  x4
};
val x5 = x2 * 5;
x5
```

Branches

Let's extend FAE with `if (e) e else e` and `<`:

```
(if (1 < 2) 1 + 2 else 3 * 4) * 5
```

```
val x1 = 1 < 2;
val x2 = if (x1) {
  val x3 = 1 + 2;
  x3
} else {
  val x4 = 3 * 4;
  x4
};
val x5 = x2 * 5;
x5
```



```
START:
  lt x1, 1, 2
  jz x1, L1
  add x3, 1, 2
  mov x2, x3
  jmp L2
L1:
  mul x4, 3, 4
  mov x2, x4
L2:
  mul x5, x2, 5
  mov r1, x5
  jmp HALT
HALT:
```

Branches

Let's extend FAE with `if (e) e else e` and `<`:

```
(if (1 < 2) 1 + 2 else 3 * 4) * 5
```

```
val x1 = 1 < 2;
val x2 = if (x1) {
  val x3 = 1 + 2;
  x3
} else {
  val x4 = 3 * 4;
  x4
};
val x5 = x2 * 5;
x5
```

⇒

```
START:
  lt x1, 1, 2
  jz x1, L1
  add x3, 1, 2
  mov x2, x3
  jmp L2
L1:
  mul x4, 3, 4
  mov x2, x4
L2:
  mul x5, x2, 5
  mov r1, x5
  jmp HALT
HALT:
```

- `jz x1, L1`: if the condition is false (0), jump to the `else` branch `L1`. Otherwise, run the `then` branch.

Branches

Let's extend FAE with `if (e) e else e` and `<`:

```
(if (1 < 2) 1 + 2 else 3 * 4) * 5
```

```
val x1 = 1 < 2;
val x2 = if (x1) {
  val x3 = 1 + 2;
  x3
} else {
  val x4 = 3 * 4;
  x4
};
val x5 = x2 * 5;
x5
```

⇒

```
START:
  lt x1, 1, 2
  jz x1, L1
  add x3, 1, 2
  mov x2, x3
  jmp L2
L1:
  mul x4, 3, 4
  mov x2, x4
L2:
  mul x5, x2, 5
  mov r1, x5
  jmp HALT
HALT:
```

- `jz x1, L1`: if the condition is false (0), jump to the `else` branch `L1`. Otherwise, run the `then` branch.
- Both branches put the result in `x2` and go to `L2`: the **continuation** of the `if`, as a label!

Contents

1. Compilers

2. Expressions

3. Functions
   - Lambda Lifting
   - Closure Conversion

4. Names and Registers
   - Alpha Renaming
   - Register Allocation
   - Optimization

5. First-Class Continuations

Recall: Continuation-Passing Style (CPS)

In **CPS**, a function does not return. It takes the **continuation** `k` and calls it with the result:

```
def double(x: Int): Int = x * 2                          // direct style
double(3) + 1

def doubleK(x: Int, k: Int => Int): Int = k(x * 2) // CPS
doubleK(3, y => y + 1)
```

Recall: Continuation-Passing Style (CPS)

PLRG

In **CPS**, a function does not return. It takes the **continuation** `k` and calls it with the result:

```
def double(x: Int): Int = x * 2                          // direct style
double(3) + 1

def doubleK(x: Int, k: Int => Int): Int = k(x * 2) // CPS
doubleK(3, y => y + 1)
```

`y => y + 1` is the **rest of the computation** after `double(3)`.

CPS Transformation

```
(x => x * 2)(3) + 1
```

1. A call must **return**. CPS makes **where to return** a function.

CPS Transformation

```
HALT(
  (x => x * 2)(3) + 1
)
```

1. A call must **return**. CPS makes **where to return** a function.
2. The result of the whole program goes to `HALT`.

CPS Transformation

```
(x => x * 2)(
  3,
  y => HALT(y + 1)
)
```

1. A call must **return**. CPS makes **where to return** a function.
2. The result of the whole program goes to `HALT`.
3. What to do **after** the call is `y => HALT(y + 1)`. Pass it to the function.

CPS Transformation

```
((x, k) => k(x * 2))(
  3,
  y => HALT(y + 1)
)
```

1. A call must **return**. CPS makes **where to return** a function.
2. The result of the whole program goes to `HALT`.
3. What to do **after** the call is `y => HALT(y + 1)`. Pass it to the function.
4. The function takes it as `k`, and gives its result to `k`.

CPS Transformation

```
((x, k) => {
  val x1 = x * 2;
  k(x1)
})(
  3,
  y => {
    val x2 = y + 1;
    HALT(x2)
  }
)
```

1. A call must **return**. CPS makes **where to return** a function.
2. The result of the whole program goes to `HALT`.
3. What to do **after** the call is `y => HALT(y + 1)`. Pass it to the function.
4. The function takes it as `k`, and gives its result to `k`.
5. Name the intermediate values (ANF). This is the **CPS** form.

Lambda Lifting

```
((x, k) => {
  val x1 = x * 2;
  k(x1)
})(
  3,
  y => {
    val x2 = y + 1;
    HALT(x2)
  }
)
```

1. The CPS form has two **nested** functions. In the IR, code must be at the **top level** with a **label**.

Lambda Lifting

```
((x, k) => {
  val x1 = x * 2;
  k(x1)
})(
  3,
  y => {
    val x2 = y + 1;
    HALT(x2)
  }
)
```

1. The CPS form has two **nested** functions. In the IR, code must be at the **top level** with a **label**.
2. First, lift `(x, k) => ...`.

Lambda Lifting

```
val F1 = (x, k) => {
  val x1 = x * 2;
  k(x1)
};
F1(
  3,
  y => {
    val x2 = y + 1;
    HALT(x2)
  }
)
```

1. The CPS form has two **nested** functions. In the IR, code must be at the **top level** with a **label**.
2. First, lift `(x, k) => ...`.
3. Move it to the **top level**, name it `F1`, and use the name where it was.

Lambda Lifting

```
val F1 = (x, k) => {
  val x1 = x * 2;
  k(x1)
};
F1(
  3,
  y => {
    val x2 = y + 1;
    HALT(x2)
  }
)
```

1. The CPS form has two **nested** functions. In the IR, code must be at the **top level** with a **label**.
2. First, lift `(x, k) => ...`.
3. Move it to the **top level**, name it `F1`, and use the name where it was.
4. Next, lift `y => ...`.

Lambda Lifting

```
val F1 = (x, k) => {
  val x1 = x * 2;
  k(x1)
};
val K1 = y => {
  val x2 = y + 1;
  HALT(x2)
};
F1(3, K1)
```

1. The CPS form has two **nested** functions. In the IR, code must be at the **top level** with a **label**.
2. First, lift `(x, k) => ...`.
3. Move it to the **top level**, name it `F1`, and use the name where it was.
4. Next, lift `y => ...`.
5. Name it `K1`. This is **lambda lifting**: no function is nested anymore.

Calls as Jumps

```
val F1 = (x, k) => {
  val x1 = x * 2;
  k(x1)
};
val K1 = y => {
  val x2 = y + 1;
  HALT(x2)
};
F1(3, K1)
```

Calls as Jumps

```
val F1 = (x, k) => {
  val x1 = x * 2;
  k(x1)
};
val K1 = y => {
  val x2 = y + 1;
  HALT(x2)
};
F1(3, K1)
```

```
F1:
  mul x1, r1, 2
  mov r1, x1
  jmp r2
K1:
  add x2, r1, 1
  mov r1, x2
  jmp HALT
START:
  mov r1, 3
  mov r2, K1
  jmp F1
HALT:
```

Calls as Jumps

```
val F1 = (x, k) => {
  val x1 = x * 2;
  k(x1)
};
val K1 = y => {
  val x2 = y + 1;
  HALT(x2)
};
F1(3, K1)
```

```
F1:
  mul x1, r1, 2
  mov r1, x1
  jmp r2
K1:
  add x2, r1, 1
  mov r1, x2
  jmp HALT
START:
  mov r1, 3
  mov r2, K1
  jmp F1
HALT:
```

- A function is a **label**. We pass arguments in `r1`, `r2`, ...
- A **call** is a **jump**: `F1(3, K1)` is `jmp F1`.
- A **return** is a jump to `k`: `k(x1)` is `jmp r2`.

Calls as Jumps

```
val F1 = (x, k) => {
  val x1 = x * 2;
  k(x1)
};
val K1 = y => {
  val x2 = y + 1;
  HALT(x2)
};
F1(3, K1)
```

```
F1:
  mul x1, r1, 2
  mov r1, x1
  jmp r2
K1:
  add x2, r1, 1
  mov r1, x2
  jmp HALT
START:
  mov r1, 3
  mov r2, K1
  jmp F1
HALT:
```

- A function is a **label**. We pass arguments in `r1`, `r2`, ...
- A **call** is a **jump**: `F1(3, K1)` is `jmp F1`.
- A **return** is a jump to `k`: `k(x1)` is `jmp r2`.

But can we **always** lift a function like this?

When Lifting Is Not Enough

Not always. Here, the inner function uses the **outer** variable `x`:

```
(x => y => x + y)(1)(2)
```

When Lifting Is Not Enough

Not always. Here, the inner function uses the **outer** variable `x`:

```
(x => y => x + y)(1)(2)
```

```
((x, k1) => {
  k1((y, k2) => {
    val z = x + y;
    k2(z)
  })
})(
  1,
  f => f(2, HALT)
)
```

When Lifting Is Not Enough

Not always. Here, the inner function uses the **outer** variable `x`:

```
(x => y => x + y)(1)(2)
```

```
((x, k1) => {
  k1((y, k2) => {
    val z = x + y;
    k2(z)
  })
})(
  1,
  f => f(2, HALT)
)
```

If we lift `(y, k2) => ...` to the top level, where does `x` come from? It is called later as `f(2, HALT)`, where `x` is **not** known anymore.

When Lifting Is Not Enough

Not always. Here, the inner function uses the **outer** variable `x`:

```
(x => y => x + y)(1)(2)
```

```
((x, k1) => {
  k1((y, k2) => {
    val z = x + y;
    k2(z)
  })
})(
  1,
  f => f(2, HALT)
)
```

If we lift `(y, k2) => ...` to the top level, where does `x` come from? It is called later as `f(2, HALT)`, where `x` is **not** known anymore.

So, the function value itself must **carry** `x`: a **closure**.

Closure Conversion

```
((x, k1) => {
  k1((y, k2) => {
    val z = x + y;
    k2(z)
  })
})(
  1,
  f => f(2, HALT)
)
```

1. `(y, k2) => ...` **captures** `x`.

Closure Conversion

```
val F1 = (y, k2) => {
  val z = x + y;
  k2(z)
};
((x, k1) => {
  k1(F1)
})(
  1,
  f => f(2, HALT)
)
```

1. `(y, k2) => ...` **captures** `x`.
2. Lift it as `F1`. But now, where does `x` come from?

Closure Conversion

```
val F1 = (y, k2) => {
  val z = x + y;
  k2(z)
};
((x, k1) => {
  k1([F1, x])
})(
  1,
  f => f(2, HALT)
)
```

1. `(y, k2) => ...` **captures** `x`.
2. Lift it as `F1`. But now, where does `x` come from?
3. Where it was, create a **closure** `[F1, x]`: an **array** with the **label** and the **captured** values (in general, `[F, x1, ..., xn]`).

Closure Conversion

```
val F1 = (y, k2) => {
  val z = x + y;
  k2(z)
};
((x, k1) => {
  k1([F1, x])
})(
  1,
  f => f[0](f, 2, HALT)
)
```

1. `(y, k2) => ...` **captures** `x`.
2. Lift it as `F1`. But now, where does `x` come from?
3. Where it was, create a **closure** `[F1, x]`: an **array** with the **label** and the **captured** values (in general, `[F, x1, ..., xn]`).
4. The caller does not know which code `f` is: jump to `f[0]`, and pass `f` itself.

Closure Conversion

```
val F1 = (c, y, k2) => {
  val x = c[1];
  val z = x + y;
  k2(z)
};
((x, k1) => {
  k1([F1, x])
})(
  1,
  f => f[0](f, 2, HALT)
)
```

1. `(y, k2) => ...` **captures** `x`.
2. Lift it as `F1`. But now, where does `x` come from?
3. Where it was, create a **closure** `[F1, x]`: an **array** with the **label** and the **captured** values (in general, `[F, x1, ..., xn]`).
4. The caller does not know which code `f` is: jump to `f[0]`, and pass `f` itself.
5. `F1` gets the closure as `c`, and reads `x` from `c[1]`.

Closure Conversion

```
val F1 = (c, y, k2) => {
  val x = c[1];
  val z = x + y;
  k2(z)
};
val F2 = (x, k1) => {
  k1([F1, x])
};
val K1 = f => {
  f[0](f, 2, HALT)
};
F2(1, K1)
```

1. `(y, k2) => ...` **captures** `x`.
2. Lift it as `F1`. But now, where does `x` come from?
3. Where it was, create a **closure** `[F1, x]`: an **array** with the **label** and the **captured** values (in general, `[F, x1, ..., xn]`).
4. The caller does not know which code `f` is: jump to `f[0]`, and pass `f` itself.
5. `F1` gets the closure as `c`, and reads `x` from `c[1]`.
6. The other two capture nothing: lift them as `F2` and `K1`.

Closure Conversion – Low-level IR

```
val F1 = (c, y, k2) => {
  val x = c[1];
  val z = x + y;
  k2(z)
};
val F2 = (x, k1) => {
  k1([F1, x])
};
val K1 = f => {
  f[0](f, 2, HALT)
};
F2(1, K1)
```

Closure Conversion – Low-level IR

```
val F1 = (c, y, k2) => {
  val x = c[1];
  val z = x + y;
  k2(z)
};
val F2 = (x, k1) => {
  k1([F1, x])
};
val K1 = f => {
  f[0](f, 2, HALT)
};
F2(1, K1)
```



```
F1:
  ld x, r1, 1
  add z, x, r2
  mov r1, z
  jmp r3
F2:
  mov x1, hp
  add hp, hp, 2
  st x1, 0, F1
  st x1, 1, r1
  mov r1, x1
  jmp r2
```

```
K1:
  ld x2, r1, 0
  mov r2, 2
  mov r3, HALT
  jmp x2
START:
  mov r1, 1
  mov r2, K1
  jmp F2
HALT:
```

Closure Conversion – Low-level IR

```
val F1 = (c, y, k2) => {
  val x = c[1];
  val z = x + y;
  k2(z)
};
val F2 = (x, k1) => {
  k1([F1, x])
};
val K1 = f => {
  f[0](f, 2, HALT)
};
F2(1, K1)
```

$\Rightarrow$

```
F1:
  ld x, r1, 1
  add z, x, r2
  mov r1, z
  jmp r3
F2:
  mov x1, hp
  add hp, hp, 2
  st x1, 0, F1
  st x1, 1, r1
  mov r1, x1
  jmp r2
```

```
K1:
  ld x2, r1, 0
  mov r2, 2
  mov r3, HALT
  jmp x2
START:
  mov r1, 1
  mov r2, K1
  jmp F2
HALT:
```

- We keep a register `hp` (**heap pointer**) pointing to **free memory**. `[F1, x]` has two elements: **allocate** two cells at `hp` (`add hp, hp, 2`), and store `F1` and `x` in them (**st**).
- `f[0]`: load the label from the closure (**ld**).

Contents

1. Compilers

2. Expressions

3. Functions
   - Lambda Lifting
   - Closure Conversion

4. Names and Registers
   - Alpha Renaming
   - Register Allocation
   - Optimization

5. First-Class Continuations

Alpha Renaming

Lifting and closure conversion move code out of its **scope**. If two different variables have the **same name**, they get mixed up:

```
val x = 1;
(x => x * 2)(3) + x
```

- The result is $6 + 1 = 7$. Without scopes, both `x`s would be **one** location: the call puts 3 in it, and the result becomes $6 + 3 = 9$. Wrong!

Alpha Renaming

Lifting and closure conversion move code out of its **scope**. If two different variables have the **same name**, they get mixed up:

```
val x = 1;
(x => x * 2)(3) + x
```

$\Rightarrow$

```
val x1 = 1;
(x2 => x2 * 2)(3) + x1
```

- The result is $6 + 1 = 7$. Without scopes, both `x`s would be **one** location: the call puts 3 in it, and the result becomes $6 + 3 = 9$. Wrong!
- **Alpha renaming** gives every variable a **unique** name **before** the transformations, without changing the meaning.

Register Allocation

PLRG

The IR uses **as many names as needed**, but a real machine has only a **few registers**. Consider this sequence:

```
mul x1, 2, 3
add x2, 1, x1
mul x3, 4, 5
add x4, x2, x3
```

- Not all four names are **alive** at the same time.

Register Allocation

The IR uses **as many names as needed**, but a real machine has only a **few registers**. Consider this sequence:

```
mul x1, 2, 3
add x2, 1, x1
mul x3, 4, 5
add x4, x2, x3
```

$\Rightarrow$

```
mul r1, 2, 3
add r1, 1, r1
mul r2, 4, 5
add r1, r1, r2
```

- Not all four names are **alive** at the same time.
- `x1` dies where `x2` is born: they **share** `r1`.

Register Allocation

The IR uses **as many names as needed**, but a real machine has only a **few registers**. Consider this sequence:

```
mul x1, 2, 3
add x2, 1, x1
mul x3, 4, 5
add x4, x2, x3
```

$\Rightarrow$

```
mul r1, 2, 3
add r1, 1, r1
mul r2, 4, 5
add r1, r1, r2
```

- Not all four names are **alive** at the same time.
- `x1` dies where `x2` is born: they **share** `r1`.
- `x3` is alive with `x2`: `r2`. `x4` reuses `r1`.

Register Allocation

The IR uses **as many names as needed**, but a real machine has only a **few registers**. Consider this sequence:

```
mul x1, 2, 3
add x2, 1, x1
mul x3, 4, 5
add x4, x2, x3
```

$\Rightarrow$

```
mul r1, 2, 3
add r1, 1, r1
mul r2, 4, 5
add r1, r1, r2
```

- Not all four names are **alive** at the same time.
- `x1` dies where `x2` is born: they **share** `r1`.
- `x3` is alive with `x2`: `r2`. `x4` reuses `r1`.
- This is **register allocation**, usually solved by **graph coloring**.

Optimization

The IR can often be made **faster** and **smaller**. For example:

- **Constant folding**: compute at compile time.

```
add x1, 2, 3
```

$\Rightarrow$

```
mov x1, 5
```

Optimization

The IR can often be made **faster** and **smaller**. For example:

- **Constant folding**: compute at compile time.

```
add x1, 2, 3
```

$\Rightarrow$

```
mov x1, 5
```

- **Dead code elimination**: `x2` is never used, and then neither is `x1`.

```
mov x1, 5
mul x2, x1, 2
mov r1, 7
jmp HALT
```

$\Rightarrow$

```
mov r1, 7
jmp HALT
```

Optimization

The IR can often be made **faster** and **smaller**. For example:

- **Constant folding**: compute at compile time.

```
add x1, 2, 3
```

$\Rightarrow$

```
mov x1, 5
```

- **Dead code elimination**: `x2` is never used, and then neither is `x1`.

```
mov x1, 5
mul x2, x1, 2
mov r1, 7
jmp HALT
```

$\Rightarrow$

```
mov r1, 7
jmp HALT
```

- **Common subexpression elimination**: reuse the same result.

```
add x1, r1, r2
add x2, r1, r2
```

$\Rightarrow$

```
add x1, r1, r2
mov x2, x1
```

Optimization

The IR can often be made **faster** and **smaller**. For example:

- **Constant folding**: compute at compile time.

```
add x1, 2, 3
```
$\Rightarrow$
```
mov x1, 5
```

- **Dead code elimination**: `x2` is never used, and then neither is `x1`.

```
mov x1, 5
mul x2, x1, 2
mov r1, 7
jmp HALT
```
$\Rightarrow$
```
mov r1, 7
jmp HALT
```

- **Common subexpression elimination**: reuse the same result.

```
add x1, r1, r2
add x2, r1, r2
```
$\Rightarrow$
```
add x1, r1, r2
mov x2, x1
```

- And many more.

Contents

1. Compilers

2. Expressions

3. Functions
   - Lambda Lifting
   - Closure Conversion

4. Names and Registers
   - Alpha Renaming
   - Register Allocation
   - Optimization

5. First-Class Continuations

First-Class Continuations

```
2 * { vcc k; 3 + k(5) }
```

1. The KFAE example in Lecture 16 ($2 * 5 = 10$).

First-Class Continuations

```
HALT(
  2 * { vcc k; 3 + k(5) }
)
```

1. The KFAE example in Lecture 16 ($2 * 5 = 10$).
2. The result of the whole program goes to `HALT`.

First-Class Continuations

```
val K1 = v => HALT(2 * v);
K1({ vcc k; 3 + k(5) })
```

1. The KFAE example in Lecture 16 ($2 * 5 = 10$).
2. The result of the whole program goes to `HALT`.
3. The block's continuation: `K1`.

First-Class Continuations

```
val K1 = v => HALT(2 * v);
val k = (v, k2) => K1(v);
K1(3 + k(5))
```

1. The KFAE example in Lecture 16 ($2 * 5 = 10$).
2. The result of the whole program goes to `HALT`.
3. The block's continuation: `K1`.
4. `vcc k`: `K1` as a function `k` that **ignores** its own continuation `k2`.

First-Class Continuations

```
val K1 = v => HALT(2 * v);
val k = (v, k2) => K1(v);
k(5, y => K1(3 + y))
```

1. The KFAE example in Lecture 16 ($2 * 5 = 10$).
2. The result of the whole program goes to `HALT`.
3. The block's continuation: `K1`.
4. `vcc k`: `K1` as a function `k` that **ignores** its own continuation `k2`.
5. `3 + k(5)` **as usual**, with the continuation `y => K1(3 + y)`.

First-Class Continuations

```
val K1 = v => {
  val x1 = 2 * v;
  HALT(x1)
};
val k = (v, k2) => K1(v);
k(5, y => {
  val x2 = 3 + y;
  K1(x2)
})
```

1. The KFAE example in Lecture 16 ($2 * 5 = 10$).
2. The result of the whole program goes to `HALT`.
3. The block's continuation: `K1`.
4. `vcc k`: `K1` as a function `k` that **ignores** its own continuation `k2`.
5. `3 + k(5)` **as usual**, with the continuation `y => K1(3 + y)`.
6. Name the intermediate values (ANF).

First-Class Continuations

```
val K1 = v => {
  val x1 = 2 * v;
  HALT(x1)
};
val F1 = (v, k2) => K1(v);
val K2 = y => {
  val x2 = 3 + y;
  K1(x2)
};
F1(5, K2)
```

1. The KFAE example in Lecture 16 ($2 * 5 = 10$).
2. The result of the whole program goes to `HALT`.
3. The block's continuation: `K1`.
4. `vcc k`: `K1` as a function `k` that **ignores** its own continuation `k2`.
5. `3 + k(5)` **as usual**, with the continuation `y => K1(3 + y)`.
6. Name the intermediate values (ANF).
7. Lift `k` and `y => ...` as `F1` and `K2`.

First-Class Continuations – Low-level IR

```
val K1 = v => {
  val x1 = 2 * v;
  HALT(x1)
};
val F1 = (v, k2) => K1(v);
val K2 = y => {
  val x2 = 3 + y;
  K1(x2)
};
F1(5, K2)
```

First-Class Continuations – Low-level IR

```
val K1 = v => {
  val x1 = 2 * v;
  HALT(x1)
};
val F1 = (v, k2) => K1(v);
val K2 = y => {
  val x2 = 3 + y;
  K1(x2)
};
F1(5, K2)
```

$\Rightarrow$

```
K1:
  mul x1, 2, r1
  mov r1, x1
  jmp HALT
F1:
  jmp K1
K2:
  add x2, 3, r1
  mov r1, x2
  jmp K1
START:
  mov r1, 5
  mov r2, K2
  jmp F1
HALT:
```

First-Class Continuations – Low-level IR

```
val K1 = v => {
  val x1 = 2 * v;
  HALT(x1)
};
val F1 = (v, k2) => K1(v);
val K2 = y => {
  val x2 = 3 + y;
  K1(x2)
};
F1(5, K2)
```

$\Rightarrow$

```
K1:
  mul x1, 2, r1
  mov r1, x1
  jmp HALT
F1:
  jmp K1
K2:
  add x2, 3, r1
  mov r1, x2
  jmp K1
START:
  mov r1, 5
  mov r2, K2
  jmp F1
HALT:
```

- `k` is lifted as `F1`: it jumps to `K1` and never to `r2`.

First-Class Continuations – Low-level IR

```
val K1 = v => {
  val x1 = 2 * v;
  HALT(x1)
};
val F1 = (v, k2) => K1(v);
val K2 = y => {
  val x2 = 3 + y;
  K1(x2)
};
F1(5, K2)
```

$\Rightarrow$

```
K1:
  mul x1, 2, r1
  mov r1, x1
  jmp HALT
F1:
  jmp K1
K2:
  add x2, 3, r1
  mov r1, x2
  jmp K1
START:
  mov r1, 5
  mov r2, K2
  jmp F1
HALT:
```

- `k` is lifted as `F1`: it jumps to `K1` and never to `r2`.
- So, `K2` $(3 + _)$ is **never** run: it is **dead code**. So is `mov r2, K2`.

First-Class Continuations – Low-level IR

```
val K1 = v => {
  val x1 = 2 * v;
  HALT(x1)
};
val F1 = (v, k2) => K1(v);
val K2 = y => {
  val x2 = 3 + y;
  K1(x2)
};
F1(5, K2)
```

$\Rightarrow$

```
K1:
  mul x1, 2, r1
  mov r1, x1
  jmp HALT
F1:
  jmp K1
START:
  mov r1, 5
  jmp F1
HALT:
```

- `k` is lifted as `F1`: it jumps to `K1` and never to `r2`.
- So, `K2` $(3 + _)$ is **never** run: it is **dead code**. So is `mov r2, K2`.
- **Dead code elimination** removes them.

Summary

PLRG

1. Compilers

2. Expressions

3. Functions
   - Lambda Lifting
   - Closure Conversion

4. Names and Registers
   - Alpha Renaming
   - Register Allocation
   - Optimization

5. First-Class Continuations

Project #2

The MiniPython language is a toy language inspired by Python.[1]

https://github.com/ku-plrg-classroom/docs/tree/main/cose212/mini-python

- Please see above document on GitHub:
  1. Implement `step` function.
  2. Implement `locals` function.

- It is just a project, and you **don't need to submit** anything.

- However, some exam questions might be related to this project, so we **strongly recommend** solving it by yourself.

- The given test cases are **not sufficient**. Please write your own test cases to test your implementation.

[1] https://www.python.org/

Next Lecture

- Type Systems

Jihyeok Park
jihyeok_park@korea.ac.kr
https://plrg.korea.ac.kr